

こどもプログラミング教室 3-1 インベーダーゲーム 改造後の解答例プログラム

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<!-- ここからJavaScriptプログラム -->
<script type="text/javascript" src="/parts.js"></script><!-- パーツ -->
<script type="text/javascript" src="/partsb.js"></script><!-- パーツB -->
<script type="text/javascript" src="/partsc.js"></script><!-- パーツC -->
<script type="text/javascript">
//固定データ(変わらないデータ)
let timer = 20 //主処理を呼ぶタイマーの実行間隔
//自機のX座標, Y座標, 幅, 高さ, 速度
let player = { x:300, y:395, sx:40, sy:40, mx:10 }
//敵機[0]のX座標, Y座標, 幅, 高さ, X速度, Y速度
let teki = { x:10, y:70, sx:60, sy:60, mx:1, my:10 }
//自弾の幅, 高さ, 速度
let tama = { sx:20, sy:20, mx:10 }
//UFOの出現X座標, Y座標, 幅, 高さ, X速度, 登場待ち
let ufo = { x:0, y:15, sx:60, sy:40, mx:2, wait:150 }
//UFO炎上の消滅待ち
let ufob = { wait:50 }
//敵弾の幅, 高さ, 速度, 発射待ち
let tekitama = { sx:20, sy:20, my:2, wait:100 }
//障害物[0]のX座標, Y座標, 幅, 高さ
let shogai = { x:40, y:350, sx:20, sy:20 }

//作業データ(ゲームの状態によって値が変わります)
let mode = 0 //ゲームの状態(0:タイトル画面,1:プレイ画面,9:ゲームオーバー画面)
let tamaf = 0 //自弾有無
let tekinum = 32 //敵機残数
let tekia = [] //敵機のリスト
let kougeki = 0 //自弾が発射されているか?
let hidari = 0 //左キーが押されている?
let migi = 0 //右キーが押されている?
let ufobw = 0 //UFO有無
let ufow = ufo.wait //UFO登場待ち
let ufobw = ufob.wait //UFO炎上の消滅待ち
let tekitamaf = 0 //敵弾有無
let tekitamaw = tekitama.wait //敵弾発射待ち
```

改造B

改造C

```
//キーが押し終わった時にやること
function keyUp(evt) {
  if (mode === 0) { //タイトル画面？
    if (evt.keyCode === 13) { //Enterキー？
      tekijunbi() //敵機全機出現
      shogaijunbi() //障害物出現
      mode = 1 //プレイ画面へ
    }
  } else if (mode === 1) { //プレイ画面？
    if (evt.keyCode === 37) { //←キー？
      hidari = 0 //左キーが離された
    }
    if (evt.keyCode === 39) { //→キー？
      migi = 0 //右キーが離された
    }
    if (evt.keyCode === 32) { //Spaceキー？
      kougeki = 0 //スペースキーが離された
    }
  }
}
```

```
//キーが押された時の処理
function keyDown(evt) {
  if (mode === 1) { //プレイ画面？
    if (evt.keyCode === 37) { //←キー？
      hidari = 1 //左キーが押された
    }
    if (evt.keyCode === 39) { //→キー？
      migi = 1 //右キーが押された
    }
    if (evt.keyCode === 32) { //Spaceキー？
      kougeki = 1 //スペースキーが押された
    }
  }
}
```

```
//ゲーム中一定時間おきに実行される処理
function mainloop() {
  if (mode === 1) { //プレイ画面？
    if (hidari === 1) { //左キーが押されている？
      hidariidou() //自機を左へ
    }
    if (migi === 1) { //右キーが押されている？
      migidou() //自機を右へ
    }
    if (kougeki === 1 && tamaf === 0) { //弾が出ていない、かつ、スペースキーが押された？
      tamauchi() //弾を発射
      tamaf = 1 //弾が出ていることにする
    }
    if (tamaf === 1) { //弾が出されている？
      tamaf = hit() //敵に当たったかどうかを調べる
    }
    if (tamaf === 1) { //弾が残っている？
      tamaf = tamashogaihit() //弾と障害物がぶつかったかどうか調べる
    }
    if (tamaf === 1) { //弾が残っている？
      tamaf = tamaidou() //弾の移動する
    }
    if (tekinum > 0) { //敵機がまだある？
      tekishogaihit() //敵機と障害物がぶつかったかどうか調べる
      tekiidou() //敵機の移動
      if (shinryaku() === 1) { //侵略された？
        mode = 9 //ゲームオーバー画面へ
      }
    }
    if (tekinum === 0) { //敵機全滅？
      tekijunbi() //敵機をもう一度出す
    }
    if (ufow === 0) { //UFOが未登場？
      if (ufow-- < 0) { //登場待ちカウントダウン後、到達？
        ufojunbi() //UFO登場
        ufow = 1 //UFO登場中
      }
    } else if (ufow === 2) { //UFOが炎上中？
      if (ufobw-- < 0) { //炎上中カウントダウン後、到達？
        ufow = 0 //UFOを消す
      }
    }
    if (ufow === 1 && tamaf === 1) { //UFOがある？
      ufow = ufohit2() //UFOと自弾の衝突判定
      if (ufow === 2) { //衝突？
        tamaf = 0 //自弾も消す
      }
    }
    if (ufow === 1) { //UFOがまだある？
      ufow = ufoido() //UFOの移動
    }
    if (tekitamaf !== 1) { //敵弾がない？（炎上中でもない？）
      if (tekitamaw-- < 0) { //発射待ちカウントダウン後、到達？
        tekikogeki() //敵弾発射
        tekitamaf = 1 //敵弾がある
      }
    }
    if (tekitamaf === 1) { //敵弾がある？
      tekitamaf = tekitamashogaihit() //敵弾と障害物がぶつかったかどうか調べる
    }
    if (tekitamaf === 1) { //敵弾がある？
      tekitamaf = tekitamatamahit() //敵弾と自弾がぶつかった？
      if (tekitamaf === 1) { //まだ敵弾がある？
        tekitamaf = tekitamaplayerhit() //敵弾と自機がぶつかった？
        tekitamaf = tekitamaido() //敵弾の移動
      } else { //でなければ
        tamaf = 0 //自弾を消す
        tekitamaf = 2 //敵弾を炎上状態にする
        tekitamaw = tekitama.wait //敵弾タイマーをセット（炎上状態）
      }
    }
  }
  draw() //画面を描く
}
```

//画面を描く処理

```
function draw() {
  if (mode === 0) { //タイトル画面？
    ctx.drawImage(back1i, 0, 0) //背景画像を表示
    ctx.font = '62pt Arial' //文字サイズを指定
    ctx.fillStyle = 'Orange' //文字をオレンジ色に
    ctx.fillText('Vertical Shooting', 10, 100) //文字列を表示
    ctx.font = '26pt Arial' //文字サイズを指定
    ctx.fillText('Hit Enter key to Start', 170, 450) //文字列を表示

    ctx.fillStyle = ctx.createPattern(tekii, 'repeat') //敵機画像でパターンを作る
    for (let i = 0; i < 5; i++) { //i=0,1,2,3,4について繰り返す
      if (i === 0) { //0番目ならば
        ctx.fillRect(240, 115, 180, 60)
      } else if (i <= 3) { //でなくて1〜3番目ならば
        ctx.fillRect(300, 115 + i * 60, 60, 60)
      } else { //でなければ
        ctx.fillRect(240, 355, 180, 60)
      }
    }
  }
  if (mode === 1) { //プレイ画面？
    ctx.drawImage(back2i, 0, 0) //背景画像を表示
    if (tamaf === 1) { //弾が出ている？
      ctx.drawImage(tamai, tamax, tamay) //指定の位置に弾を表示
    }
    if (ufof === 1) { //UFO登場中？
      ctx.drawImage(ufoi, ufox, ufoy) //指定の位置にUFO画像を表示
    } else if (ufof === 2) { //UFO炎上中？
      ctx.drawImage(ufobi, ufox, ufoy) //指定の位置にUFO炎上画像を表示
    }
    if (tekitamaf === 1) { //敵弾が有る？
      ctx.drawImage(tekitamai, tekitamax, tekitamay) //指定の位置に敵弾画像を表示
    } else if (tekitamaf === 2) { //敵弾が炎上状態？
      ctx.drawImage(enjo2i, tekitamax, tekitamay) //指定の位置に敵弾炎上画像を表示
    }
  }
  if (mode === 9) { //ゲームオーバー画面？
    ctx.drawImage(back1i, 0, 0) //背景画像を表示
    ctx.font = '90pt Arial' //文字サイズを指定
    ctx.fillStyle = 'Red' //赤色に設定
    ctx.fillText('Game Over', 10, 150) //文字列を表示
    ctx.font = '26pt Arial' //文字サイズを指定
    ctx.fillText('Reload to Re-Start', 190, 450) //文字列を表示
    ctx.drawImage(enjoi, playerx - 10, playery - 10) //指定の位置に炎上画像を表示
  }
  if (mode !== 0) { //タイトル画面以外？
    ctx.drawImage(playeri, playerx, playery) //指定の位置に自機画像を表示
    for (let i = 0; i < 32; i++) { //敵機全機について繰り返す
      if (tekia[i].f === 1) { //敵機[i]が有る？
        ctx.drawImage(tekii, tekia[i].x, tekia[i].y) //敵機[i]の画像を表示
      }
    }
    for (let i = 0; i < 24; i++) { //障害物全てについて繰り返す
      if (shogaia[i].f === 1) { //障害物[i]が有る？
        ctx.drawImage(shogaai, shogaia[i].x, shogaia[i].y) //障害物[i]の画像を表示
      }
    }
    ctx.font = '15pt Arial' //フォントサイズを指定
    ctx.fillStyle = 'Orange' //オレンジ色に設定
    let txt = '000000' + score //スコアの前に000000を連結
    ctx.fillText('Score:' + txt.substring(txt.length - 6), 10, 20) //スコアの下6桁を表示
  }
}
```

改造A

```

</script>
<!-- ここまでJavaScriptプログラム -->
</head>
<body onload="init();" > <!-- 始まる時の処理 -->
<canvas id="canvas" width="640" height="480" /></canvas> <!-- キャンバスの設置 -->
 <!-- タイトル背景画像を非表示で指定 -->
 <!-- ゲーム背景画像を非表示で指定 -->
 <!-- 自機画像を非表示で指定 -->
 <!-- 敵機画像を非表示で指定 -->
 <!-- 自弾画像を非表示で指定 -->
 <!-- UFO画像を非表示で指定 -->
 <!-- UFO炎上画像を非表示で指定 -->
 <!-- 敵弾画像を非表示で指定 -->
 <!-- 自機炎上画像を非表示で指定 -->
 <!-- 敵弾炎上画像を非表示で指定 -->
 <!-- 障害物画像を非表示で指定 -->
</body>
</html>

```